

Project Report: Stock Identification from Order Book Data

Emmanuel Gardin

April 13, 2025

Contents

1	Problem Presentation	2
2	Codebase Structure	2
3	Data Processing Plan	3
4	Large Data Engineering and Management	4
5	Data Visualization	4
6	Data Preprocessing (Pipelines and Transformers)	5
6.1	Feature Engineering + Gradient Boosting (GB) Approach	6
6.2	Deep Learning (GRU) Approach	6
7	Model Architectures	6
7.1	Gradient Boosting Model (OrderBookGBModel)	7
7.2	Recurrent GRU Model (OrderBookGRUModel)	7
8	Training and Hyperparameter Optimization	8
9	Results and Analysis	8
9.1	Gradient Boosting (GB) Model Results	8
9.2	GRU Model Results	10
10	API Implementation	12
11	Conclusion and Perspectives	12

1 Problem Presentation

This project addresses the challenge proposed by Capital Fund Management (CFM) on the Challengedata platform. The main objective is to identify the original stock from a sequence of tick-by-tick order book events. This is framed as a **multi-class classification problem** with 24 distinct stock categories (labeled 0 to 23).

The provided data consists of sequences, each containing 100 consecutive events for a single observation (`obs_id`). Each event includes information such as:

- **venue**: Integer identifier for the exchange where the event occurred (categorical).
- **action**: Type of order book event ('A' - Add, 'D' - Delete, 'U' - Update) (categorical).
- **order_id**: Unique (obfuscated) identifier to track a specific order (categorical/identifier).
- **side**: Side of the order book ('A' - Ask/Offer, 'B' - Bid) (categorical).
- **price**: Price of the affected order (numerical).
- **bid**: Best available bid price after the event (numerical).
- **ask**: Best available ask price after the event (numerical).
- **bid_size**: Total volume available at the best bid price (numerical).
- **ask_size**: Total volume available at the best ask price (numerical).
- **flux**: Change in volume caused by the event (numerical).
- **trade**: Boolean indicator signaling if the event was due to a trade (categorical/boolean).

The raw data mixes **numerical** (prices, volumes) and **categorical** (venue, action, side, trade) information, meeting the project requirements.

The challenge lies in the anonymized nature of the data and the normalization of absolute prices, preventing trivial identification. Therefore, subtle patterns and characteristics specific to each stock (volatility, typical spread, trade frequency, volume distribution, etc.) must be extracted for successful classification. The **sequential** nature of the data (100 events per observation) is a key feature to leverage.

2 Codebase Structure

The project code is organized modularly within a Python package named `gts_challenge`, facilitating maintenance, reuse, and extension. The main structure is as follows:

```
.
├── data/
│   ├── preprocessed_data/
│   │   ├── X_test.parquet
│   │   ├── X_train.parquet
│   │   └── y_train.parquet
└── gts_challenge/
    ├── api/
    ├── order_book/
    │   ├── base/
    │   └── data/
```

```

        models/
        utils/
        workflows/
models/
  gb/
  gru/
notebooks/
  project.ipynb
  sample_payload_*.json
.gitignore
launch_api.bat
readme.md
requirements.txt

```

This structure adheres to good development practices:

- **Modularity:** Clear separation of concerns.
- **Interfaces:** Use of abstract classes (`ModelInterface`, `PipelineInterface`).
- **Factory:** Centralized creation of workflows (`factory.py`).
- **Data Management:** Dedicated modules.
- **Workflows:** Encapsulated specific logic.
- **Dedicated API:** Separation for deployment.

3 Data Processing Plan

Given the sequential nature and mixed data types, two distinct methodological approaches were explored:

1. Feature Engineering + Gradient Boosting Approach:

- *Principle:* Transform each sequence into a vector of aggregated features summarizing the observed behavior, then train a tabular model (LightGBM).
- *Advantages:* Potential interpretability, robust models on tabular data.
- *Disadvantages:* Potential loss of fine-grained temporal information.
- *Implementation:* `GBPipeline` and `OrderBookGBModel`.

2. Deep Learning Approach with Recurrent Networks (GRU):

- *Principle:* Use a model (GRU) capable of natively processing sequences to automatically learn relevant representations.
- *Advantages:* Potential capture of complex temporal patterns, no heavy manual feature engineering.
- *Disadvantages:* More complex models ("black box"), need for more data/computation.
- *Implementation:* `GRUPipeline` and `OrderBookGRUModel`.

This dual approach allows comparing the effectiveness of the two paradigms.

4 Large Data Engineering and Management

The significant volume of the initial data (several GB) required specific strategies:

1. **Format Conversion:** Use of the **Parquet** format (`gts_challenge/order_book/data/loaders.py`) for raw tabular data (more efficient than CSV).
2. **Optimized Storage:**
 - **HDF5** format (`gts_challenge/order_book/data/data_utils.py`) for preprocessed features (NumPy arrays), allowing batch access.
 - Parquet format for labels (smaller).
3. **Batch Generation (Chunking):** Development of a generator (`PreprocessedDataGenerator` in `gts_challenge/order_book/data/generators.py`) reading preprocessed data in batches during training, avoiding loading everything into RAM.
4. **Preprocessing Saving/Loading:** Pipelines and preprocessed data are saved to avoid repeating lengthy computations during subsequent runs (`gts_challenge/order_book/data/data_utils.py`, `gts_challenge/order_book/workflows/*_workflow.py`).

This data engineering effort was crucial due to the large scale of the data and the limited computational resources available on a personal laptop, making the project significantly focused on efficient data handling.

5 Data Visualization

Before modeling, an exploratory data analysis was performed on the training data (`X_train.parquet`, `y_train.parquet`) using the `project.ipynb` notebook. Key observations include:

- **Target Distribution (`eqt_code_cat`):** The number of sequences per stock category in the training dataset is perfectly balanced, with an equal representation for each of the 24 classes.
- **Venue Distribution:** Events are not uniformly distributed across venues. Venues 4 and 5 are predominant, indicating higher activity or larger market share for these exchanges in the dataset.
- **Action Distribution:** Add ('A') and Delete ('D') actions are the most frequent, suggesting significant order placement and cancellation activity compared to updates ('U').
- **Side Distribution:** The distribution of events affecting the bid ('B') versus the ask ('A') side of the order book appears balanced.
- **Trade Distribution:** The vast majority of order book updates (deletions or updates) are not caused by trades (`trade=False`). This aligns with known market microstructure behavior where cancellations are much more frequent than executions.
- **Correlation Heatmap:** A heatmap of numerical features (calculated on a sample) revealed expected correlations, such as between the `ask` price and the event `price`. Some correlation was also observed between `ask_size` and `bid_size`. Most other feature pairs exhibited relatively low correlation.

These visualizations helped understand the data's characteristics and informed the modeling choices.

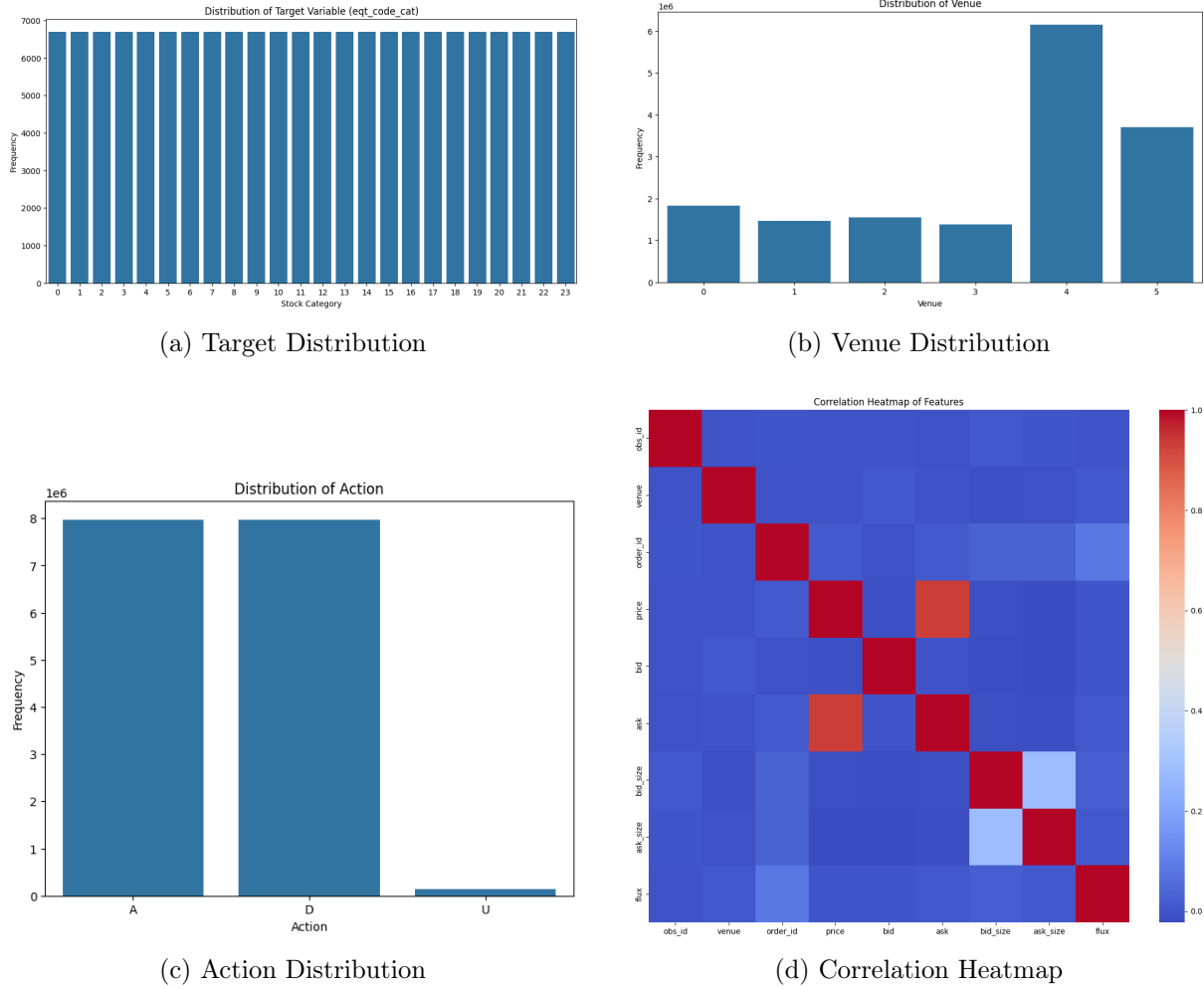


Figure 1: Data Visualization Summary (Placeholders)

6 Data Preprocessing (Pipelines and Transformers)

Two distinct pipelines, incorporating custom scikit-learn compatible transformers, were developed, fulfilling the project requirement for custom transformers within pipelines.

Common Custom Transformers:

- **NonPositiveBidSizeFilter** (`gts_challenge/order_book/data/preprocessors.py`): *Transformer 1 (Cleaning)*. Filters observations with `bid_size ≤ 0`.
- **CategoricalEncoder** (`gts_challenge/order_book/data/preprocessors.py`): *Transformer 2 (Simple Encoding)*. Numerically encodes action and side.
- **DataVectorizer** (`gts_challenge/order_book/data/preprocessors.py`): *Transformer 3 (Vectorization)*. Encodes venue, action, trade; applies $\log_{1p}$ to `bid_size`, `ask_size`; transforms flux (sign-preserving log).
- **VectorizedSequenceReshaper** (`gts_challenge/order_book/data/preprocessors.py`): *Transformer 4 (Shaping)*. Groups by `obs_id` and creates NumPy sequence tensors (shape $(n_obs, seq_len, n_features)$ or (n_obs, seq_len)).

6.1 Feature Engineering + Gradient Boosting (GB) Approach

The GBPipeline (`gts_challenge/order_book/models/gb/pipeline.py`) chains the common transformers and then adds aggregated feature extraction.

Table 1: Steps of the GBPipeline

Step	Transformer	Role
1	NonPositiveBidSizeFilter	Data cleaning (<code>bid_size ≤ 0</code>).
2	CategoricalEncoder	Numerical encoding of <code>action</code> and <code>side</code> .
3	DataVectorizer	Encoding of <code>venue</code> , <code>trade</code> , log transformations.
4	VectorizedSequenceReshaper	Sequence shaping (prerequisite).
5	GBFeatureExtractor	Extraction of aggregated features.
(6)	StandardScaler (internal)	Standardization of extracted features.

The GBFeatureExtractor (`gts_challenge/order_book/models/gb/pipeline.py`) is the *Transformer 5 (Advanced Feature Engineering)*. It computes 15 features summarizing each sequence, as selected in the notebook:

- `price_trend_ratio`, `return_volatility`, `norm_price_range`, `rel_price_momentum`, `return_autocorr` (Price/Return based)
- `mean_rel_spread`, `rel_spread_volatility`, `spread_trend` (Spread based)
- `log_volume_imbalance`, `volume_imbalance_stability`, `volume_trend` (Volume based)
- `action_transition_rate`, `action_diversity`, `mean_log_flux`, `norm_flux_volatility` (Activity based)

The code was designed flexibly to allow easy selection of features for the pipeline via the "gb features" parameter.

6.2 Deep Learning (GRU) Approach

The GRUPipeline (`gts_challenge/order_book/models/gru/pipeline.py`) is more direct.

Table 2: Steps of the GRUPipeline

Step	Transformer	Role
1	NonPositiveBidSizeFilter	Data cleaning (<code>bid_size ≤ 0</code>).
2	CategoricalEncoder	Numerical encoding of <code>action</code> and <code>side</code> .
3	DataVectorizer	Encoding of <code>venue</code> , <code>trade</code> , log transformations.
4	VectorizedSequenceReshaper	Tensor shaping (final GRU format).

The result is a dictionary of NumPy tensors directly usable by the GRU model.

7 Model Architectures

Two distinct models, conforming to `ModelInterface`, were implemented.

7.1 Gradient Boosting Model (OrderBookGBModel)

This model (`gts_challenge/order_book/models/gb/model.py`) uses **LightGBM**. It takes the 15 engineered features from the **GBPipeline** as input.

Table 3: Architecture and Characteristics of the **OrderBookGBModel**

Aspect	Description
Algorithm	LightGBM (<code>lgbm.LGBMClassifier</code>).
Type	Gradient Boosting Decision Trees (GBDT).
Objective	Multi-class Classification (<code>multiclass</code> , 24 classes).
Input	Dictionary of 15 aggregated features (from <code>GBFeatureExtractor</code>). Converted to <code>DataFrame</code> internally.
Hyperparameters	Configured via dictionary (see notebook): <code>learning_rate=0.05</code> , <code>n_estimators=200</code> , <code>num_leaves=31</code> , <code>feature_fraction=0.8</code> , <code>bagging_fraction=0.8</code> , etc.
Training	<code>fit</code> with <code>eval_set</code> and early stopping callback.
Output	Class predictions (0-23) or probabilities.
Interpretability	Feature importance via <code>feature_importances_</code> .

7.2 Recurrent GRU Model (OrderBookGRUModel)

This model (`gts_challenge/order_book/models/gru/model.py`), implemented with **TensorFlow/Keras**, processes the sequences prepared by **GRUPipeline**. A Gated Recurrent Unit (GRU) is a type of recurrent neural network (RNN) adept at learning from sequential data. It uses gating mechanisms (update and reset gates) to control the flow of information, allowing it to capture dependencies over time more effectively than simple RNNs while being computationally less intensive than LSTMs. This makes it suitable for analyzing the temporal patterns in the order book event sequences.

Table 4: Architecture of the **OrderBookGRUModel**

Layer	Type	Details (based on tuned params)
Inputs	Input (x4)	Categorical (<code>venue</code> , <code>action</code> , <code>trade</code>) shape (100,), Numerical shape (100, 6).
Embeddings	Embedding (x3)	<code>venue</code> ($\rightarrow$ 8 dims), <code>action</code> ($\rightarrow$ 8 dims), <code>trade</code> ($\rightarrow$ 8 dims).
Concatenation (Features)	Concatenate (axis=2)	Combine embeddings + 6 numerical features (shape (100, 30)).
Recurrent Layers	GRU (Bidirectional)	2 GRUs (forward/backward) with 128 units each, dropout $\approx$ 0.11.
Concatenation (GRU)	Concatenate	Combine forward/backward GRU outputs (shape (256,)).
Dense Layer 1	Dense	64 neurons, activation <code>selu</code> .
Output Layer	Dense	24 neurons (classes), activation <code>softmax</code> .
Compilation	-	Loss: <code>sparse_categorical_crossentropy</code> , Optimizer: <code>Adam</code> (<code>lr</code> $\approx$ 0.009), Metric: <code>accuracy</code> .

8 Training and Hyperparameter Optimization

Training is managed by a modular workflow system via `gts_challenge/order_book/factory.py`.

1. Workflow Orchestration (`factory.py`):

- `train_workflow(...)`: Executes the full flow: preprocessing $\rightarrow$ model instantiation $\rightarrow$ training. Handles data chunking and pipeline saving/loading.
- `predict_workflow(...)`: Loads trained model/pipeline and performs predictions.
- `explain_workflow(...)`: Loads model/pipeline and performs feature importance explanation.

2. Specific Training:

- **GRU** (`gru_workflow.train_model`): Training over epochs using batches from `PreprocessedDataGenerator`. Evaluation and checkpointing occur at the end of each epoch. This batching was essential due to the memory-intensive nature of GRU training on large sequences.
- **GB** (`gb_workflow.train_model`): Single-pass LightGBM training on aggregated features, using built-in validation and early stopping.

3. Hyperparameter Optimization (`gts_challenge/order_book/utils/tune.py`):

- Uses the **Optuna** library.
- The script defines an **objective** function executing `train_workflow` with hyperparameters suggested by Optuna for each trial.
- Validation performance (loss) is returned to Optuna to guide the search for optimal parameters.
- Best parameters found for GRU (used in Table 4) were: `learning_rate=0.00896`, `gru_units=128`, `dense_units=64`, `gru_dropout=0.114`.

9 Results and Analysis

Model evaluation was performed on a held-out validation set (20% of the training data). Accuracy and classification reports were the primary metrics.

9.1 Gradient Boosting (GB) Model Results

- **Performance:** The GB model achieved a validation accuracy of approximately **22%**. While this is significantly better than random guessing ($1/24 \approx 4.2\%$), it indicates limitations in capturing the necessary patterns from the engineered features alone.
- **Confusion Matrix:** The confusion matrix showed a diffuse pattern, indicating frequent misclassifications across many categories, without strong diagonal dominance.

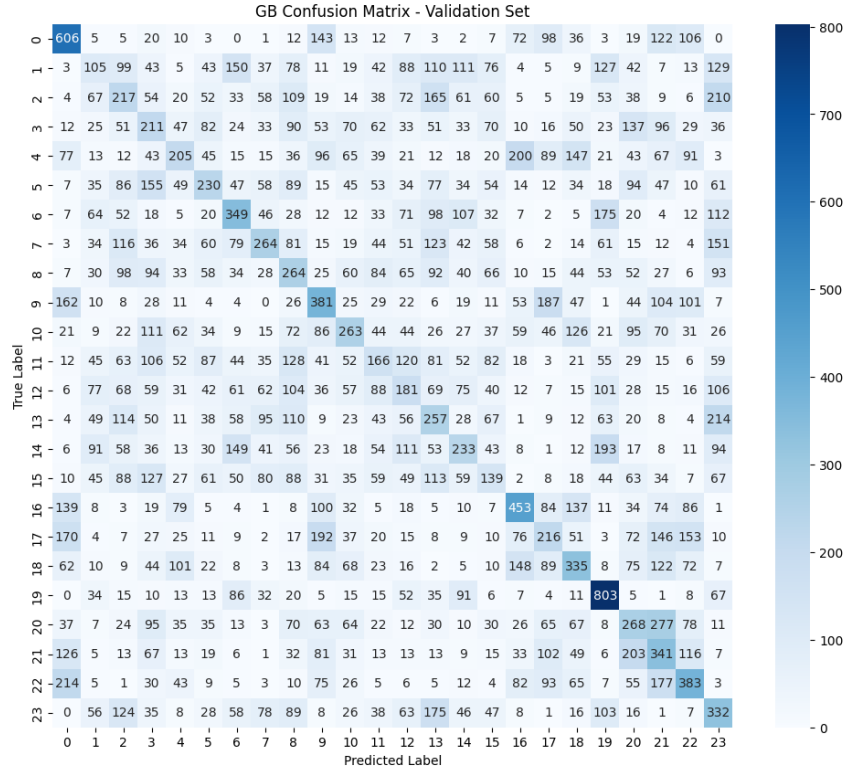


Figure 2: GB Model Confusion Matrix (Validation Set - Placeholder)

- **Feature Importance:** LightGBM's internal feature importance highlighted the significance of spread-related features (`mean_rel_spread`, `rel_spread_volatility`), return volatility (`return_volatility`), and volume imbalance (`log_volume_imbalance`).

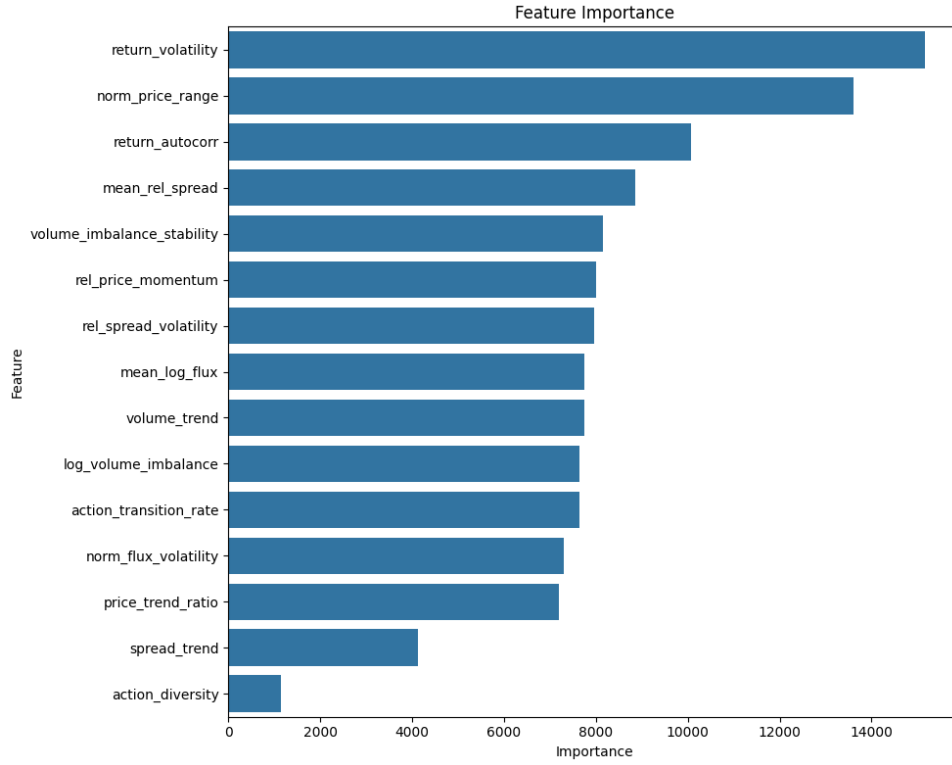


Figure 3: GB Feature Importance (LightGBM Built-in - Placeholder)

- **Interpretation:** Volatility and spread seem to be the biggest indicators of the model prediction. This could be a problem if the market conditions change a lot as volatility and spread could be changed across assets.

9.2 GRU Model Results

- **Performance:** After hyperparameter tuning with Optuna, the GRU model achieved a significantly higher validation accuracy, reaching approximately **50%**. This demonstrates the effectiveness of the recurrent architecture in learning from the sequential data.
- **Confusion Matrix:** The confusion matrix for the GRU model showed a much clearer diagonal structure compared to the GB model, indicating better classification performance across most classes, although some confusion between specific stock pairs remained.

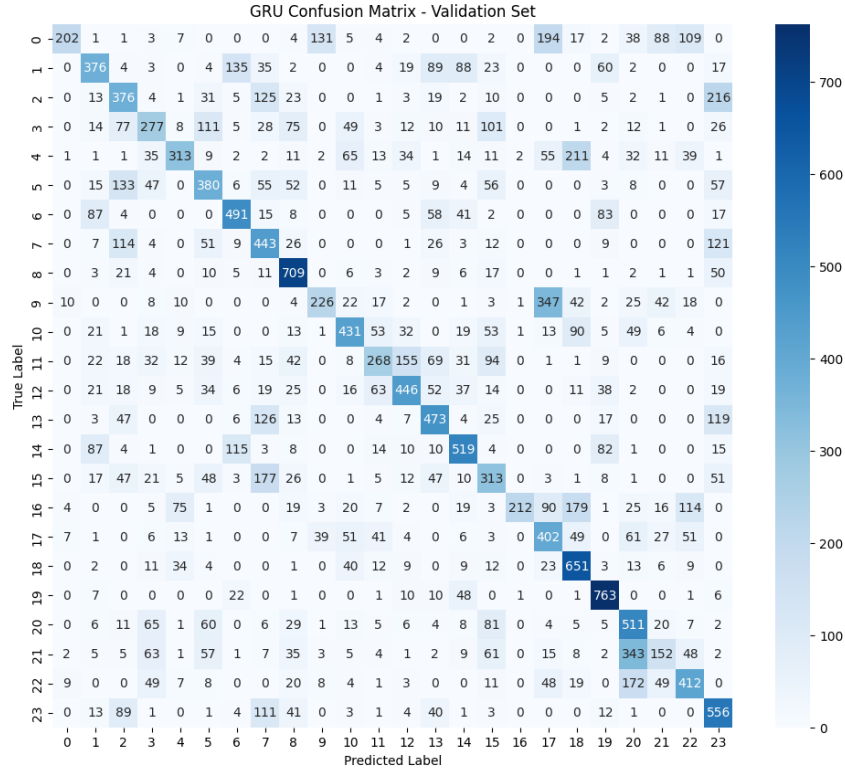


Figure 4: GRU Model Confusion Matrix (Validation Set - Placeholder)

- **Feature Importance (Permutation):** Feature importance analysis using intra-observation permutation (via the `explain_gru` workflow) indicated that the embeddings for `venue` and `action` were highly influential, along with numerical features like `log_flux` and `log_bid_size`. This suggests the model learned meaningful representations from both categorical and numerical inputs over time.

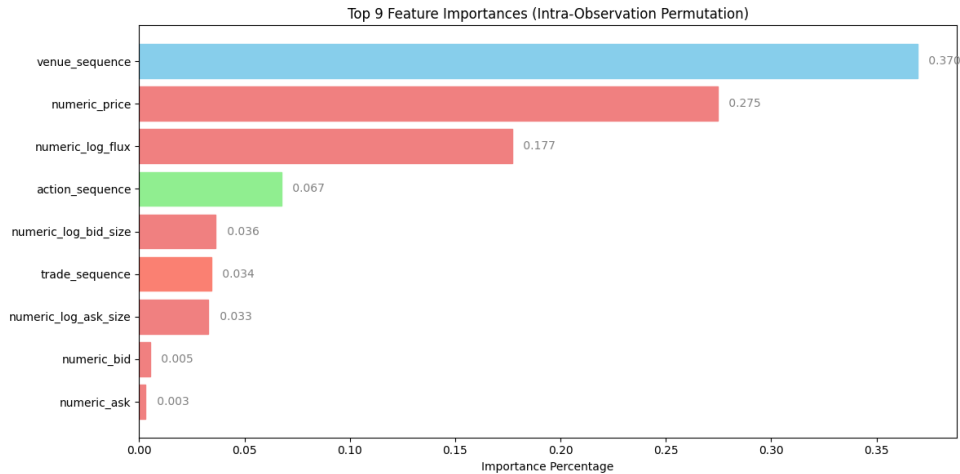


Figure 5: GRU Feature Importance (Intra-Observation Permutation - Placeholder)

- **Interpretation:** The venue is the biggest indicator. It makes sense as some stocks could be more traded on some exchanges than some others. Price and flux are the second biggest indicators. That makes sense as well – it classifies stocks based on their volatility / price evolution and also on the size of transactions.

10 API Implementation

As required, a REST API was developed using FastAPI (`gts_challenge/api/`) to serve the trained models.

- **Endpoints:** It provides `/predict` and `/explain` endpoints.
- **Input Validation:** Pydantic models (`PredictionRequest`, `ExplanationRequest`) define the expected input structure and are used for automatic data validation (a sequence of 100 events and the desired `model_type` - 'gb' or 'gru'). Sample payloads (`sample_payload_gb.json`, `sample_payload_gru.json`) were generated for testing.
- **Functionality:** The API loads the appropriate pre-trained model and pipeline based on the request's `model_type`, preprocesses the input sequence, and returns either the predicted stock category (and probabilities) or feature importance scores.
- **Deployment:** A batch script (`launch_api.bat`) simplifies setting environment variables (model/pipeline paths) and launching the Uvicorn server.

11 Conclusion and Perspectives

This project successfully explored and compared two distinct approaches—feature engineering with Gradient Boosting and sequence modeling with GRUs—for identifying stocks from order book data. The **GRU model demonstrated significantly better performance**, achieving 50% validation accuracy compared to the GB model's 22%, highlighting the critical importance of capturing temporal dynamics for this task.

A substantial effort was dedicated to **data engineering** to handle the large dataset size within limited computational resources, involving efficient data formats (Parquet, HDF5) and batch processing (`PreprocessedDataGenerator`). The codebase was designed with **modularity** in mind, utilizing interfaces, factories, and separate workflows, which facilitated the implementation and comparison of the two modeling strategies and the development of the API. Custom scikit-learn compatible transformers were integrated into distinct preprocessing pipelines for each model type.

The final GRU model provides a strong baseline for the task. The project successfully met the requirements, including developing custom transformers, pipelines, comparing models, analyzing results with explanations, and delivering a functional API with input validation using Pydantic schemas.

Future Work ("Next Steps"):

1. **Alternative Architectures:** Leverage the modular structure to test LSTMs or Transformers.
2. **Preprocessing Refinement:** Explore alternative normalization or encoding techniques.
3. **Hybrid Models:** Investigate combining GRU outputs with select engineered features from the GB approach.
4. **Advanced Explainability:** Apply methods like SHAP (carefully, given sequence length) or attention mechanisms (if using Transformers) for deeper GRU interpretation.
5. **Test Set Evaluation:** Formally evaluate the final models on the unseen test set (`X_test.parquet`) if labels were available.

In conclusion, this project delivered a robust and extensible system for stock identification, demonstrating the power of sequence modeling for financial time series and emphasizing the practical importance of data engineering in real-world machine learning applications.